Школа анализа данных

Lecture 1.2

Tools
MCP

Zaytseva Alena,
AI Lead at Yandex Lavka

Contents

Tools

01 LLM without tools

02 What are tools

03 How tool-calling works

04 Tool Design Principles

MCP

01 When is protocol needed

02 What is MCP

03 Architecture and roles

04 What problems are solved

05 About over-engineering

Practice

Adding tools to our agent

LLMs without Tools

1. Have limited capabilities due to the specifics of their training
2. Do not have access to up-to-date information
3. Cannot take actions on their own

```
=== Counting characters task ===
87

Ground truth: 97
```

```
=== Math ===
8293

Ground truth: 825.4444444444445
```

```
=== Up-to-date info ===
I don't have access to real-time data, including the current Bitcoin price.
```

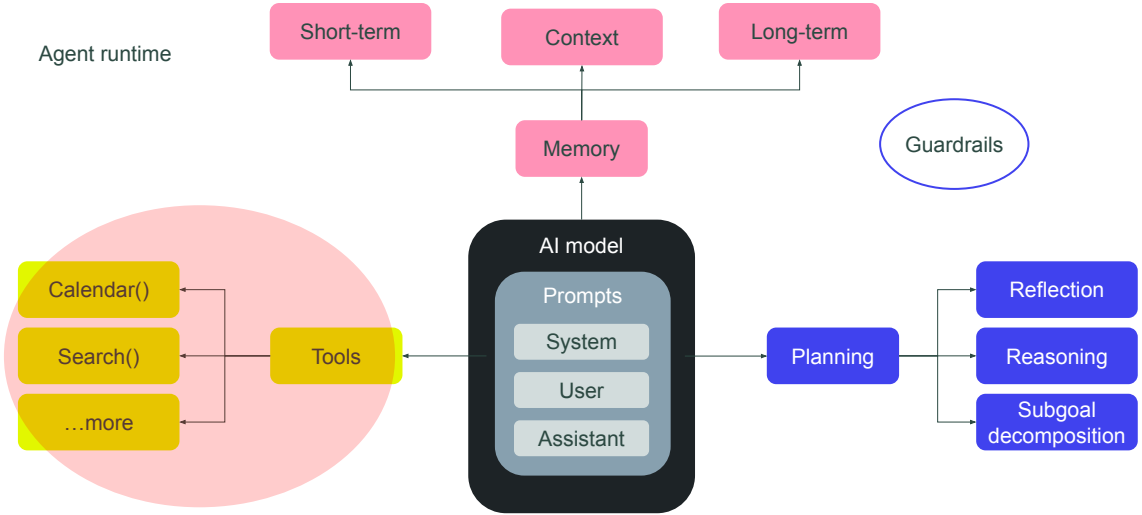
```
USER: 1) Cancel reservation EHGLP3.
AGENT: I'm unable to access specific reservations or cancel them directly.
```

What Are Tools

A tool is a function that an LLM can call to interact with external systems

A tool should include:

- What it does (text description)
- Executable function (runtime implementation)
- Input arguments (with types)
- Output (with type)



How Tool-Calling Works

LLMs only generate text — they do not execute code

How to Provide Tools to LLM

Providing tools to an LLM means:

- The model is informed about available tools – via the system prompt
- The model learns to generate tool calls when needed through:
  - Models that support tool calling are trained on tool-usage examples
    - Using the required format (e.g., special tokens)
  - Understanding your tool descriptions
    - Tool descriptions must be clear, precise, and consistent.

Tool Design Principles

Handling Partial Failures

Tools must handle partial failures while keeping the system consistent

charge payment → reserve inventory → create delivery

Idempotency & Safe Retries

Tools must safely handle repeated calls (after crashes, timeouts, agent restarts)

Sending an email → gets timeout → retry must not send it twice

Clear Error Messages

Errors must be understandable for LLMs so the agent can decide what to do next

Good errors enable the agent to:

- Ask the user for missing parameters
- Retry or offer alternatives during temporary outages

Guidance for Next Steps

Tools can return guidance that helps the agent continue the conversation properly.

Examples

- Ask the user for a delivery date
- Request a phone number in the correct format

Structured Outputs

Return structured, machine-readable outputs instead of free-form text.

Examples

- {status, order_id, delivery_window}
- Analytics returning aggregates instead of prose

Key Takeaway

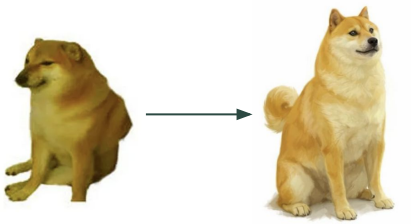
A good tool for an AI agent is not just a function

It is a reliable contract between the LLM and external systems, designed to withstand failures, retries, and changing environments.

When is Protocol Needed

Yes,

Tools have significantly expanded the capabilities of LLMs and enabled them to act as agents

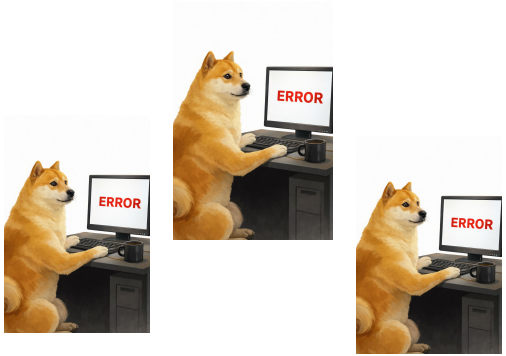


But

- Tools are embedded inside each agent
- APIs change over time
- Different models expect tools/ invoke them in different formats

→

- Tool reusability remains low
- Any API change requires updating multiple agents
- Systems become fragile and hard to scale



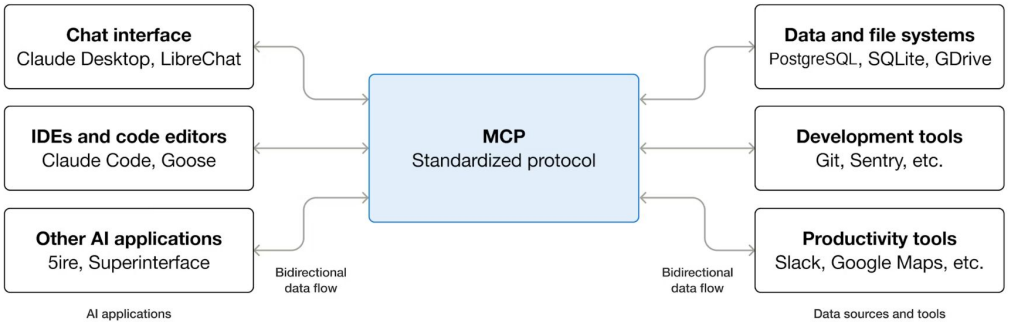
What is MCP

On November 25, 2024, Anthropic open-sourced the Model Context Protocol (MCP)

Model Context Protocol (MCP) is an open standard that enables secure, bidirectional connections between AI models and external systems (APIs, tools, data sources)

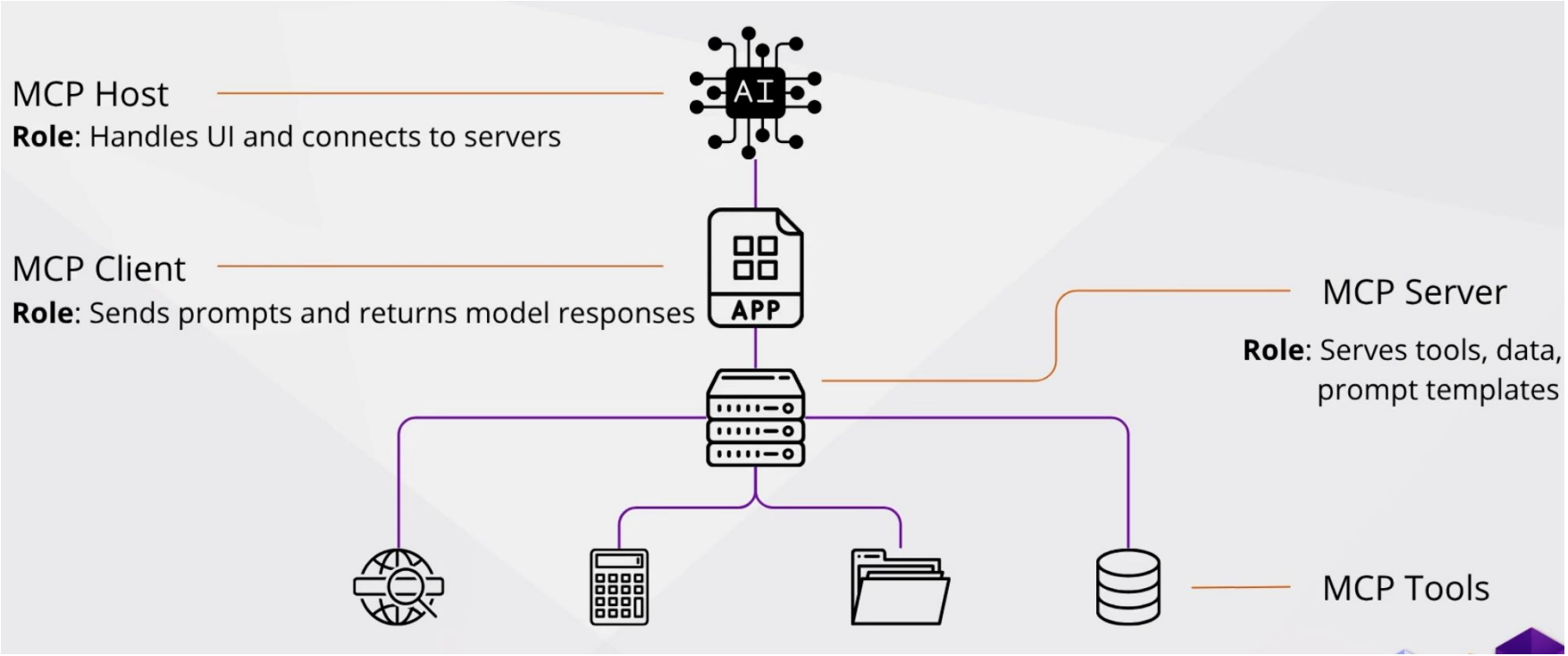
https://www.anthropic.com/news/model-context-protocol
https://modelcontextprotocol.io/docs/getting-started/intro

MCP is the "USB-C port" or "HTTP layer" of the AI ecosystem.



Architecture & Roles

Client–server architecture with three roles



Roles: Host

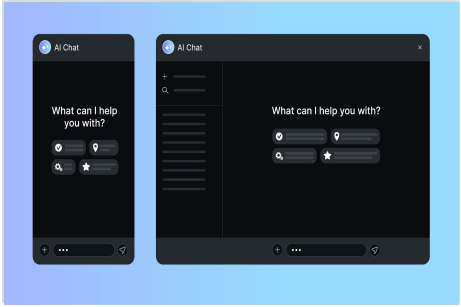
Orchestrates LLMs and AI workflows

Manages connections: one MCP client per MCP server

Controls UI: User-facing application (e.g., Cursor, custom agents)

Security & permissions: auth, limits, policies

User consent: approvals for data sharing & tool use



Roles: Server

Server = provider of context and capabilities

- A lightweight program exposing tools, context, and capabilities
- Can run locally or remotely

Tasks:

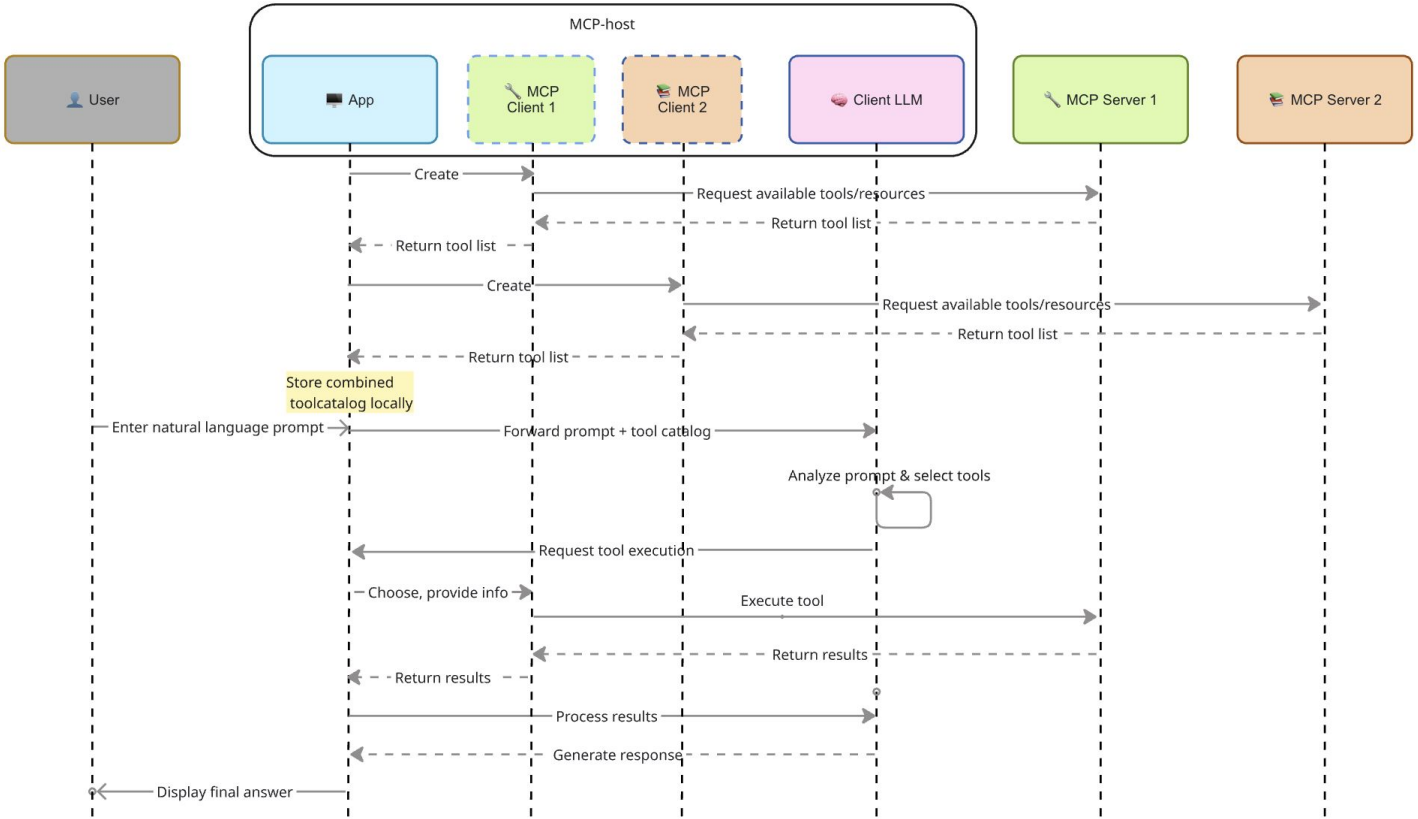
- Registers capabilities
- Processes requests
- Providing context
- Client connection handling
- Sends updates/notifications to clients

Roles: Client

Client = connector between Host and Server

- Protocol communication: JSON-RPC 2.0 requests, secure communication channels
- Capability negotiation: features & versions during initialization
- Tool execution: sends tool calls, returns results
- Real-time updates: handles server notifications

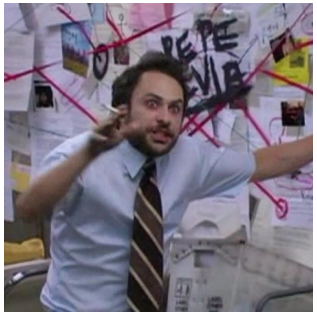
MCP Flow


MCP-host
User
App
MCP Client 1
MCP Client 2
Client LLM
MCP Server 1
MCP Server 2
Create
Request available tools/resources
Return tool list
Return tool list
Create
Request available tools/resources
Return tool list
Return tool list
Store combined toolcatalog locally
Enter natural language prompt
Forward prompt + tool catalog
Analyze prompt & select tools
Request tool execution
Choose, provide info
Execute tool
Return results
Return results
Process results
Generate response
Display final answer

Recap: problems solved

Problem	Without MCP	With MCP
Tool reusability	Each team reimplemented the same tools independently.	Tools live in MCP servers — standalone infrastructure components (like microservices). Write once → reuse across many agents and products.
API updates & maintenance	All agent teams had to update their integrations manually.	Responsibility shifts to MCP server maintainers. Agents automatically discover updated capabilities during initialization.
Scaling and modifications	Components were tightly coupled. Changing one component required rewriting many parts in each agent	Separation of responsibilities. Changes are localized to individual components.

VIBE



But

Do not over-engineer. Start simple. Add MCP when complexity forces you.

In the middle ground you can use lightweight tool ecosystems
(e.g., AgentSkills) that provide:

- Ready-to-use skills
- Standardized interfaces
- Faster development without heavy infrastructure
- A stepping stone toward more structured architectures like MCP

https://agentskills.io/home

Ready-to-use Tools

- https://docs.langchain.com/oss/python/integrations/tools
- https://github.com/punkpeye/awesome-mcp-servers
- https://zapier.com/mcp
- https://www.tavily.com/

Школа анализа данных

Thank you!

Zaytseva Alena